

# Documento de Análisis y Diseño (DAD)

---

## Plumita: aplicación web para el control de alimentación en producción avícola de traspatio

---

**Elaborado por:** Keila Valesca Ramírez

**Carnet:** 7690-22-2239

**Revisado por:** Melvin Cali

**Curso:** Seminario de Tecnologías de Información

**Fecha:** 21/08/2026

**Versión:** 1.0

---

## CONTENIDO

---

### Control de versiones

#### 1. Introducción

#### 2. Contexto general de la solución

##### 2.1. Participantes

##### 2.2. Flujo funcional de Plumita

##### 2.3. Alcance

##### 2.4. Limitaciones

#### 2.5. Cronograma de actividades

##### 2.5.1. Tabla de actividades

##### 2.5.2. Diagrama de Gantt

#### 3. Descripción del proceso de solución

##### 3.1. Requisitos funcionales (RF)

##### 3.2. Requisitos no funcionales (RNF)

##### 3.3. Dependencias

##### 3.4. Descripción de las pantallas / unidades funcionales

##### 3.5. Diagrama y descripción del proceso técnico

###### 3.5.1. Actualización automática de la edad

###### 3.5.2. Cálculo de la cantidad de alimento

#### 4. Diseño de la base de datos

##### 4.1. Estructura de tablas

##### 4.2. Diagrama entidad-relación

#### 5. Test

##### 5.1. Casos de prueba

6. Glosario

7. Anexos

7.1. Mockups de pantallas

Control de versiones

| Fecha      | Versión | Autor                 | Descripción            |
|------------|---------|-----------------------|------------------------|
| 21/08/2026 | 1.0     | Keila Valesca Ramirez | Creación del documento |

1. Introducción

Este documento explica cómo está pensado por dentro el sistema Plumita, una aplicación web enfocada en resolver un problema bien concreto: el control del alimento que se les da a las aves en una granja pequeña de traspatio. La idea nace de una situación real, porque quien maneja la granja lleva todo el registro de forma manual, sin ningún control exacto de cuánto alimento se compra, cuánto se consume ni cuánto cuesta mantener a las aves.

Con Plumita se busca que esa persona pueda registrar desde su celular la alimentación diaria de sus aves, sin necesidad de hacer cálculos a mano, y que el sistema le diga automáticamente cuánto darles según el tipo y la edad de cada ave. También se busca que tenga un control claro del inventario de alimento y de cuánto necesita comprar antes de que se le acabe.

Este documento describe el contexto general de la solución, los requisitos que debe cumplir, el diseño técnico detrás de cada funcionalidad y cómo está estructurada la información dentro del sistema.

2. Contexto general de la solución

2.1. Participantes

| Rol               | Descripción                                                                                |
|-------------------|--------------------------------------------------------------------------------------------|
| Usuaría final     | Mi mamá Linda , es la dueña de la granja avícola. Es quien registra la alimentación diaria |
| Desarrollador     | Estudiante Keila Ramirez encargada del diseño, desarrollo y pruebas del sistema como p     |
| Aves de la granja | No son usuarias del sistema, pero son el objeto central sobre el que se calcula toda la in |

2.2. Flujo funcional de Plumita

Se detalla a continuación:

La usuaria ingresa a la aplicación desde la aplicación de su celular.

Registra o consulta las aves que tiene activas, agrupadas por tipo (gallina, gallo, pato) y por etapa (pollito o adulto).

El sistema calcula automáticamente, todos los días, la edad actual de cada ave a partir de la fecha aproximada en que ingresó a la granja.

Cuando llega la hora de alimentar, el sistema le muestra la cantidad recomendada, calculada según el tipo, la etapa y la cantidad de aves que tiene.

La usuaria registra que ya alimentó a las aves, y ese registro queda guardado como parte del historial.

Cada vez que compra alimento, lo registra con la cantidad en libras y el precio pagado, y el sistema actualiza el inventario disponible.

Con base en el consumo, el sistema le proyecta cuánto alimento va a necesitar y cuánto le va a costar en un periodo de tiempo definido.

Si el alimento está por agotarse, el sistema le envía una notificación por Telegram al número de celular que registró.

## 2.3. Alcance

El sistema Plumita cubre únicamente el módulo de control de alimentación de una granja avícola de traspatio. Dentro de este alcance se incluye:

- Registro y gestión de aves por tipo y etapa.
- Cálculo automático diario de la edad de cada ave.
- Cálculo de la cantidad de alimento a dar según una tabla alimenticia parametrizada.
- Registro y consulta del historial de alimentación.
- Registro de compras de alimento y control del inventario disponible.
- Proyección de cuánto alimento se necesita comprar en un periodo definido, junto con el costo estimado.
- Envío de notificaciones por Telegram.
- Interfaz mobile-first, pensada para usarse desde el navegador de un celular.

## 2.4. Limitaciones

Así como se definió qué sí cubre el sistema, también es importante dejar claro qué queda fuera de este proyecto:

- No incluye otros módulos de gestión de granja como el control sanitario o el financiero general, únicamente el de alimentación.
- No calcula fórmulas nutricionales propias; la tabla alimenticia que usa el sistema es una tabla ya validada y se carga durante el desarrollo, no es editable por la usuaria.
- No pide ni calcula la fecha exacta de nacimiento de las aves, sino que trabaja con una fecha aproximada de ingreso a la granja.
- No se desarrolla como aplicación nativa para tiendas de aplicaciones, sino como una aplicación web accesible desde el navegador.
- No sustituye la asesoría de un médico veterinario o zootecnista.

- No tiene un nivel de disponibilidad garantizado tipo empresarial; su funcionamiento se valida durante el periodo de pruebas con la usuaria real.

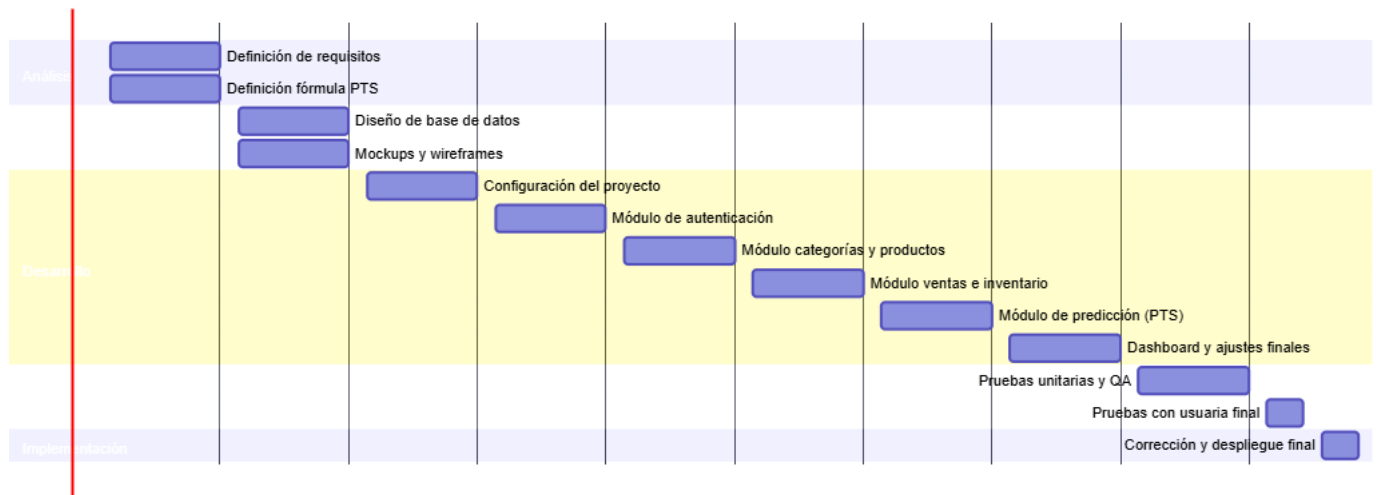
## 2.5 Cronograma de actividades

El desarrollo de StockPredict se organiza siguiendo las etapas propias del ciclo de vida del software: análisis, diseño, desarrollo, pruebas e implementación. La fecha límite para tener la aplicación implementada y funcionando es el **31 de octubre**.

### Tabla de actividades

| Etapa          |  | Actividad                                                       |  | Fecha inicio | Fech |
|----------------|--|-----------------------------------------------------------------|--|--------------|------|
| Análisis       |  | Definición de requisitos funcionales y no funcionales           |  | 24/08        | 30/0 |
| Análisis       |  | Definición de la fórmula de predicción (PTS)                    |  | 24/08        | 30/0 |
| Diseño         |  | Diseño de base de datos y diagrama entidad-relación             |  | 31/08        | 06/0 |
| Diseño         |  | Diseño de mockups y wireframes de las pantallas                 |  | 31/08        | 06/0 |
| Desarrollo     |  | Configuración del proyecto (backend, frontend, base de datos)   |  | 07/09        | 13/0 |
| Desarrollo     |  | Módulo de autenticación (registro, login, recuperar contraseña) |  | 14/09        | 20/0 |
| Desarrollo     |  | Módulo de categorías y productos (CRUD)                         |  | 21/09        | 27/0 |
| Desarrollo     |  | Módulo de ventas e inventario (compras y ajuste de stock)       |  | 28/09        | 04/1 |
| Desarrollo     |  | Módulo de predicción de stock (fórmula PTS)                     |  | 05/10        | 11/1 |
| Desarrollo     |  | Dashboard y ajustes visuales generales                          |  | 12/10        | 18/1 |
| Pruebas        |  | Pruebas unitarias y casos de prueba (QA)                        |  | 19/10        | 25/1 |
| Pruebas        |  | Pruebas con la usuaria final                                    |  | 26/10        | 28/1 |
| Implementación |  | Corrección de observaciones y despliegue final                  |  | 29/10        | 31/1 |

### Diagrama de Gantt



### 3. Descripción del proceso de solución

#### 3.1. Requisitos funcionales (RF)

##### RF-01. Registro de usuario

El sistema debe permitir crear una cuenta nueva solicitando usuario, contraseña y número de celular. El número de celular se usa después para enviar las notificaciones de alimentación.

##### RF-02. Inicio de sesión

El sistema debe permitir que la usuaria inicie sesión con su usuario y contraseña.

##### RF-03. Editar información de la cuenta

Desde la pantalla de configuración, la usuaria debe poder editar su número de celular y su contraseña.

##### RF-04. Ingresar ave

El sistema debe permitir registrar una nueva ave, seleccionando el tipo (gallina, gallo o pato), ingresando una fecha aproximada de ingreso a la granja, la edad estimada que tenía en ese momento y una breve descripción.

##### RF-05. Editar ave

El sistema debe permitir editar la fecha de ingreso, la edad estimada y la descripción de un ave ya registrada, en caso de que se haya ingresado un dato incorrecto.

##### RF-06. Inactivar ave

El sistema debe permitir inactivar un ave en vez de eliminarla, para conservar su historial de alimentación.

##### RF-07. Configurar veces de alimentación al día

El sistema debe permitir ingresar y editar cuántas veces al día se le da de comer a las aves.

##### RF-08. Calcular cantidad de alimento

El sistema debe calcular automáticamente cuánto alimento darle a las aves, tomando en cuenta la cantidad de aves según su tipo y etapa, la tabla alimenticia parametrizada y la cantidad de veces al día que se alimenta.

##### RF-09. Registrar alimentación

El sistema debe permitir registrar que se alimentó a las aves, guardando la hora del registro como parte de una bitácora.

#### **RF-10. Ver historial de alimentación**

El sistema debe mostrar un historial con las alimentaciones registradas anteriormente.

#### **RF-11. Registrar compra de alimento**

El sistema debe permitir registrar una compra de alimento, ingresando la cantidad comprada en libras y el precio pagado en quetzales.

#### **RF-12. Ver historial de compras**

El sistema debe mostrar un historial con las compras de alimento registradas.

#### **RF-13. Mostrar inventario disponible**

El sistema debe mostrar cuánto alimento se tiene disponible en libras, calculado a partir de lo comprado menos lo consumido.

#### **RF-14. Proyectar necesidad de compra**

El sistema debe proyectar cuánta cantidad de alimento se va a necesitar en un periodo de tiempo definido y cuánto costaría esa compra, usando como base el consumo promedio y el precio por libra registrado.

#### **RF-15. Enviar notificación de alimentación**

El sistema debe enviar una notificación por Telegram al celular registrado, avisando la hora de alimentar y la cantidad recomendada.

#### **RF-16. Enviar notificación de inventario bajo**

El sistema debe enviar una notificación por Telegram cuando el inventario de alimento esté por agotarse.

### **3.2. Requisitos no funcionales (RNF)**

#### **RNF-01. Diseño mobile-first**

El sistema debe estar diseñado principalmente para usarse desde el navegador de un celular, ya que es el único dispositivo que utiliza la usuaria final.

#### **RNF-02. Tiempo de carga**

Las pantallas principales deben cargar en menos de 2 segundos.

#### **RNF-03. Actualización automática de edad**

El sistema debe actualizar todos los días la edad de cada ave, tomando como base la fecha de ingreso registrada, para mantener correcto el conteo por etapa y el cálculo de alimento.

#### **RNF-04. Tabla alimenticia parametrizada**

El sistema debe contar con una tabla alimenticia organizada por tipo de ave y etapa (pollito o adulto). Esta tabla se carga durante el desarrollo del sistema y no puede ser editada por la usuaria.

#### **RNF-05. Bitácora de alimentación**

Cada registro de alimentación debe guardar la hora exacta en la que se realizó, para mantener un historial confiable.

#### **RNF-06. Validación de precio de compra**

Al registrar una compra, el sistema debe guardar el precio por libra pagado, ya que ese valor se usa como base para proyectar el costo de la próxima compra.

#### **RNF-07. Margen de error en la proyección**

La proyección de alimento necesario debe incluir un porcentaje de margen configurable, para cubrir los casos en los que se da un poco más de alimento del recomendado.

#### **RNF-08. Disponibilidad durante pruebas**

El sistema debe mantenerse disponible durante el periodo de pruebas con la usuaria real, sin que esto implique una garantía formal de disponibilidad a nivel empresarial.

### **3.3. Dependencias**

#### **Telegram (Bot API)**

El sistema depende de la API de Telegram para el envío de notificaciones. Es necesario que la usuaria vincule su número de celular con el bot configurado, para que las alertas de alimentación e inventario le lleguen correctamente.

#### **Servicio de hosting web**

El sistema depende de un servicio de hosting para que la aplicación web esté disponible durante el periodo de pruebas con la usuaria.

### **3.4. Descripción de las pantallas / unidades funcionales**

---

#### **Login**

Pantalla de entrada al sistema. Solicita usuario y contraseña. Si las credenciales son correctas, la usuaria pasa a la pantalla de inicio; si no, el sistema muestra un mensaje de error.

#### **Registro**

Permite crear una cuenta nueva. Solicita usuario, contraseña y número de celular. El número de celular queda guardado como el destino de las notificaciones de Telegram.

#### **Inicio**

Es el punto de partida después de iniciar sesión. Muestra un resumen general: cuántas aves activas se tienen, cuánto alimento hay disponible en inventario, para cuántos días alcanza ese alimento y a qué hora es la próxima alimentación con su cantidad recomendada. Esta pantalla le da a la usuaria una idea rápida del estado de la granja sin tener que entrar a otra sección.

#### **Mis aves**

Muestra las aves activas agrupadas por tipo (gallina, gallo, pato) y por etapa (pollito o adulto). Desde aquí se puede agregar una nueva ave llenando un formulario con tipo, fecha aproximada de ingreso, edad estimada en ese momento y descripción. También se puede editar o inactivar una ave ya existente.

#### **Alimentar**

Muestra la cantidad de alimento que corresponde dar en ese momento, ya calculada por el sistema. Tiene un

botón para registrar que se alimentó a las aves, lo cual queda guardado en el historial con la hora exacta. Debajo se muestra el historial reciente de alimentaciones.

### **Inventario**

Muestra cuánto alimento hay disponible en libras. Incluye una proyección de cuánto alimento se necesita comprar para cubrir un periodo definido y el costo estimado de esa compra. Tiene un botón para registrar una nueva compra (cantidad en libras y precio pagado) y muestra el historial de compras anteriores.

### **Configuración**

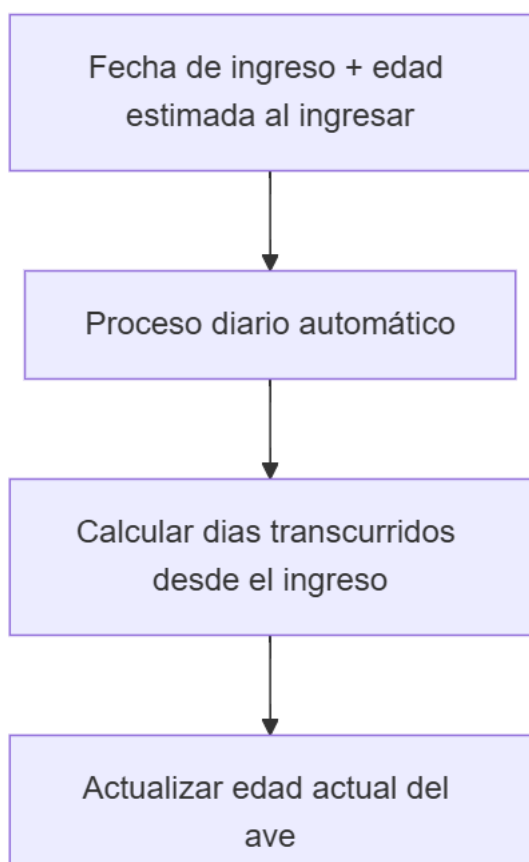
Permite editar el número de celular y la contraseña de la cuenta, así como la cantidad de veces al día que se alimenta a las aves. También muestra, solo para consulta, la tabla alimenticia parametrizada por tipo de ave y etapa.

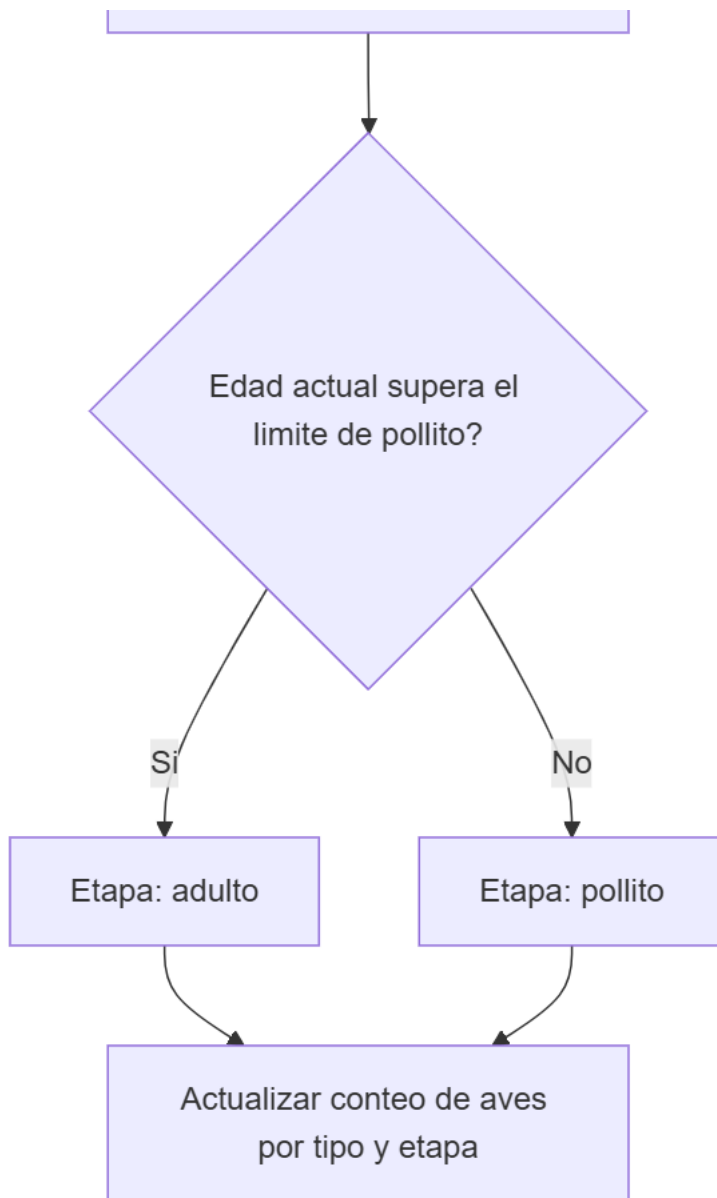
## **3.5. Diagrama y descripción del proceso técnico**

---

### **3.5.1. Actualización automática de la edad**

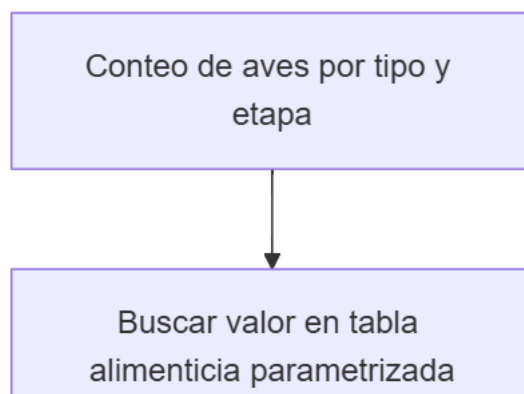
Cada ave guarda una fecha aproximada de ingreso a la granja y la edad estimada que tenía en ese momento. A partir de esos dos datos, el sistema calcula todos los días cuántos días o meses han pasado, y con eso actualiza la edad actual de cada ave. De esa edad depende directamente en qué etapa está clasificada el ave (pollito o adulto), lo cual afecta el cálculo de alimento.

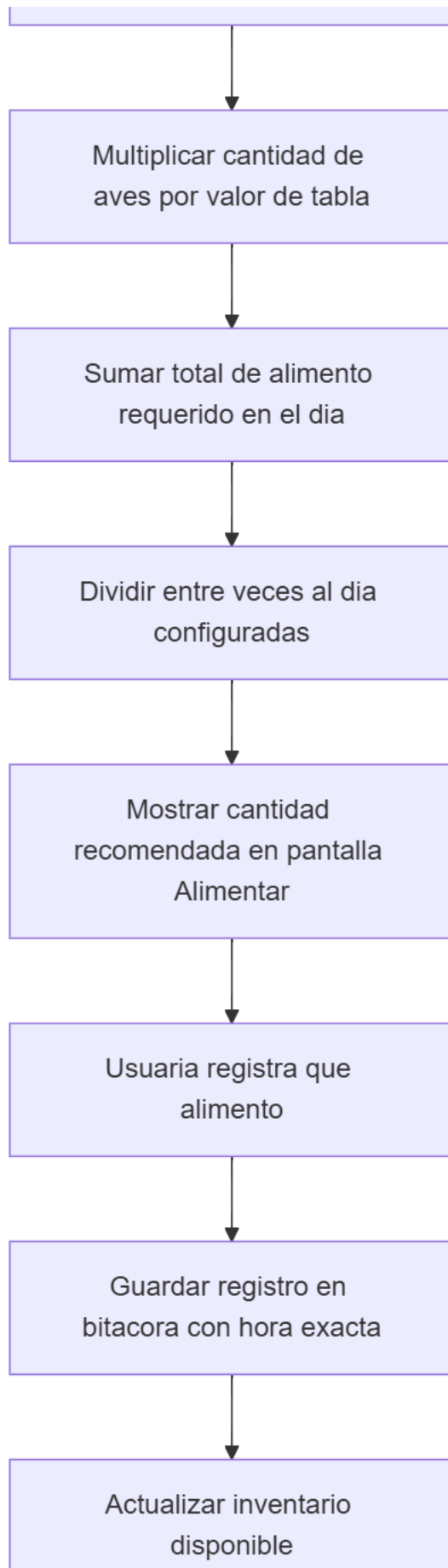


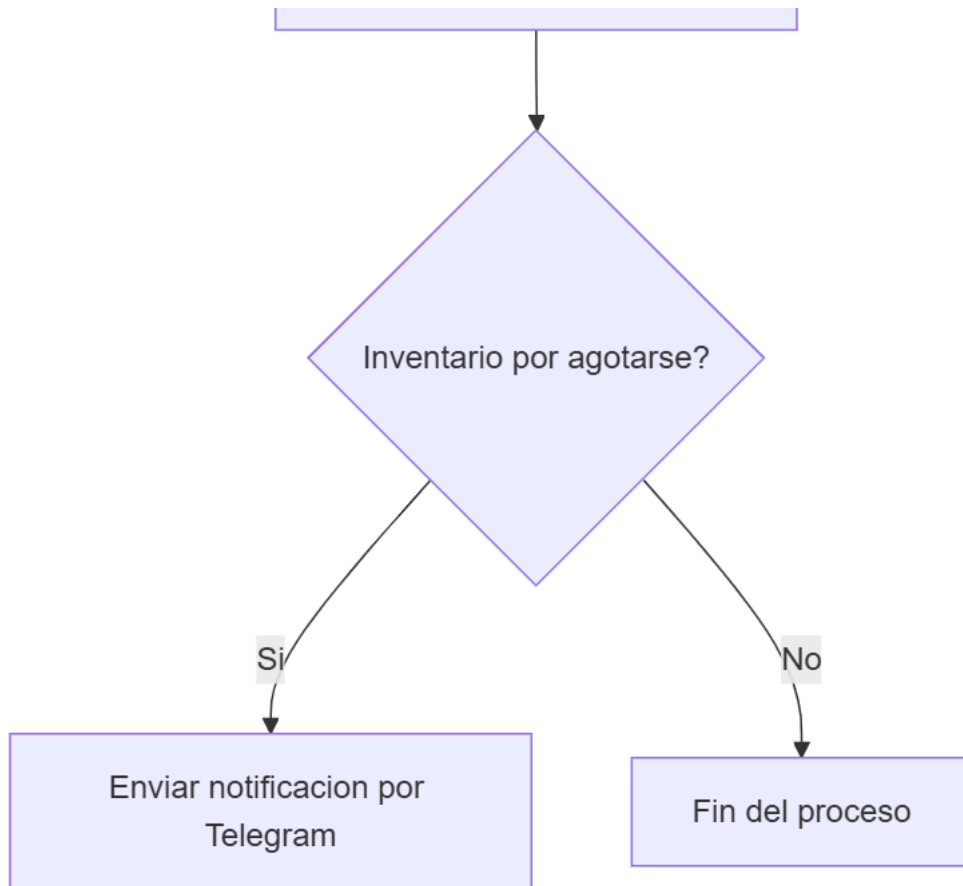


### 3.5.2. Cálculo de la cantidad de alimento

Cuando llega la hora de alimentar, el sistema toma la cantidad de aves agrupadas por tipo y etapa, busca en la tabla alimenticia parametrizada cuánto le corresponde a cada grupo, y divide ese total entre la cantidad de veces al día que se alimenta. El resultado es la cantidad exacta que la usuaria debe dar en ese momento.







## 4. Diseño de la base de datos

---

### 4.1. Estructura de tablas

#### usuario

Guarda la información de la cuenta de la usuaria y su configuración general.

| Campo                  | Tipo     | Descripción                                         |
|------------------------|----------|-----------------------------------------------------|
| id_usuario             | INT (PK) | Identificador único del usuario                     |
| nombre_usuario         | VARCHAR  | Usuario para iniciar sesión                         |
| contraseña             | VARCHAR  | Contraseña encriptada                               |
| celular                | VARCHAR  | Número de celular para notificaciones por Telegram  |
| veces_alimentacion_dia | INT      | Cantidad de veces al día que se alimenta a las aves |

#### tipo\_ave

Catálogo fijo con los tipos de ave que maneja el sistema.

| Campo       | Tipo     | Descripción                            |
|-------------|----------|----------------------------------------|
| id_tipo_ave | INT (PK) | Identificador único del tipo de ave    |
| nombre      | VARCHAR  | Nombre del tipo (gallina, gallo, pato) |

#### **ave**

Guarda cada ave registrada por la usuaria.

| Campo                       | Tipo     | Descripción                                                   |
|-----------------------------|----------|---------------------------------------------------------------|
| id_ave                      | INT (PK) | Identificador único del ave                                   |
| id_usuario                  | INT (FK) | Usuaría dueña del ave                                         |
| id_tipo_ave                 | INT (FK) | Tipo de ave según catálogo                                    |
| fecha_ingreso               | DATE     | Fecha aproximada en la que el ave ingresó a la granja         |
| edad_estimada_ingreso_meses | INT      | Edad aproximada que tenía el ave al momento de ingresar       |
| edad_actual_meses           | INT      | Edad actual, recalculada automáticamente cada día             |
| etapa_actual                | VARCHAR  | Etapas actual del ave (pollito o adulto), derivada de la edad |
| descripcion                 | VARCHAR  | Descripción breve del ave                                     |
| activo                      | BOOLEAN  | Indica si el ave sigue activa o fue inactivada                |

#### **tabla\_alimenticia**

Tabla parametrizada con la cantidad de alimento recomendada. Se carga durante el desarrollo y no es editable por la usuaria.

| Campo                | Tipo     | Descripción                                         |
|----------------------|----------|-----------------------------------------------------|
| id_tabla_alimenticia | INT (PK) | Identificador único del registro                    |
| id_tipo_ave          | INT (FK) | Tipo de ave al que aplica                           |
| etapa                | VARCHAR  | Etapas a la que aplica (pollito o adulto)           |
| cantidad_alimento_lb | DECIMAL  | Cantidad de alimento recomendada por ave, en libras |

#### **registro\_alimentacion**

Bitácora de cada vez que se registró una alimentación.

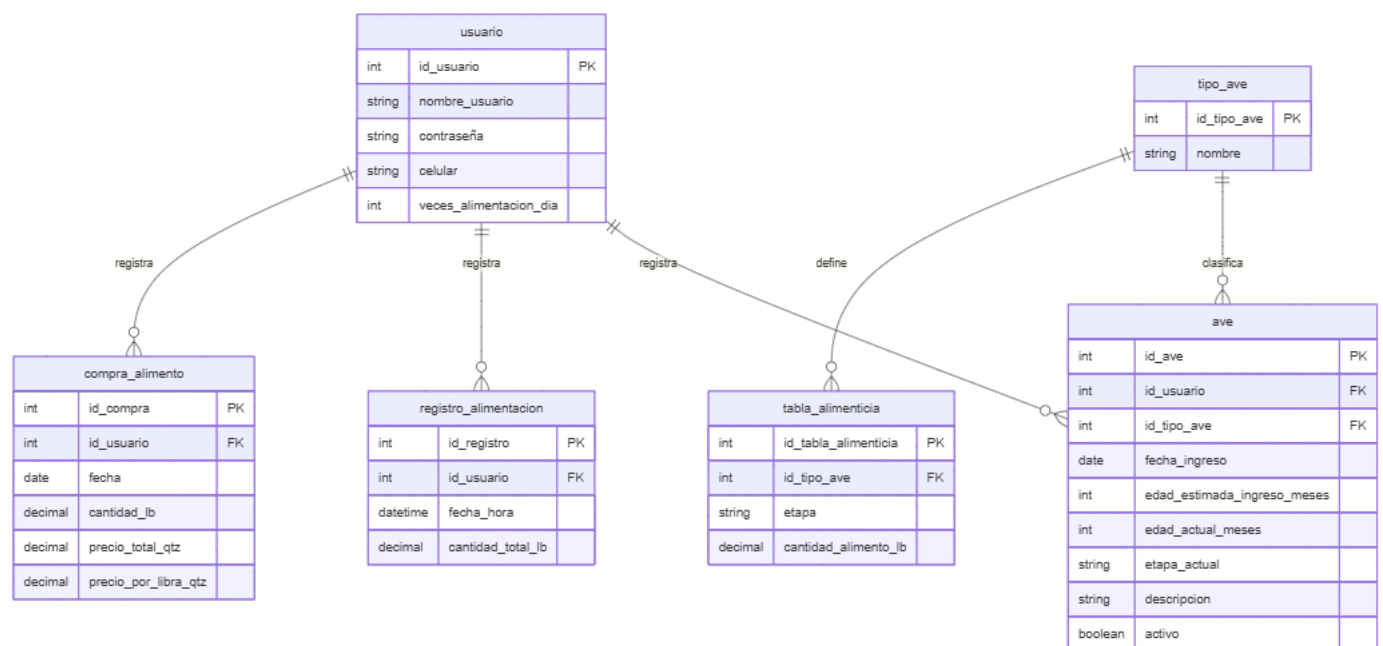
| Campo             | Tipo     | Descripción                                     |
|-------------------|----------|-------------------------------------------------|
| id_registro       | INT (PK) | Identificador único del registro                |
| id_usuario        | INT (FK) | Usuarua que hizo el registro                    |
| fecha_hora        | DATETIME | Fecha y hora exacta en la que se alimentó       |
| cantidad_total_lb | DECIMAL  | Cantidad total de alimento dado en ese registro |

### compra\_alimento

Guarda cada compra de alimento registrada.

| Campo                | Tipo     | Descripción                                    |
|----------------------|----------|------------------------------------------------|
| id_compra            | INT (PK) | Identificador único de la compra               |
| id_usuario           | INT (FK) | Usuarua que registró la compra                 |
| fecha                | DATE     | Fecha de la compra                             |
| cantidad_lb          | DECIMAL  | Cantidad comprada, en libras                   |
| precio_total_qtz     | DECIMAL  | Precio total pagado, en quetzales              |
| precio_por_libra_qtz | DECIMAL  | Precio por libra, calculado a partir del total |

## 4.2. Diagrama entidad-relación



Tomando en cuenta que el inventario disponible no se guarda como una tabla aparte: se calcula sumando lo registrado en `compra_alimento` y restando lo registrado en `registro_alimentacion`. Así se evita duplicar información y el dato siempre queda actualizado según los movimientos reales.

## 5. Test

### 5.1. Casos de prueba

| ID    | Caso de prueba                   | Entrada                                                     |
|-------|----------------------------------|-------------------------------------------------------------|
| CP-01 | Registro de usuario              | Usuario, contraseña y celular válidos                       |
| CP-02 | Inicio de sesión correcto        | Usuario y contraseña correctos                              |
| CP-03 | Inicio de sesión incorrecto      | Usuario o contraseña incorrectos                            |
| CP-04 | Ingresar ave                     | Tipo de ave, fecha de ingreso y edad estimada               |
| CP-05 | Actualización automática de edad | Ave registrada con fecha de ingreso pasada                  |
| CP-06 | Cambio de etapa por edad         | Ave que alcanza la edad límite de pollito                   |
| CP-07 | Inactivar ave                    | Ave activa seleccionada para inactivar                      |
| CP-08 | Cálculo de alimento              | Cantidad de aves por tipo/etapa y veces al día configuradas |
| CP-09 | Registrar alimentación           | Confirmación de alimentación desde la pantalla "Alimentar"  |
| CP-10 | Registrar compra de alimento     | Cantidad en libras y precio en quetzales                    |
| CP-11 | Proyección de compra             | Consumo histórico y margen de error configurado             |
| CP-12 | Notificación de inventario bajo  | Inventario por debajo del umbral definido                   |
| CP-13 | Edición de configuración         | Cambio de celular o contraseña                              |

## 7. Glosario

| Término                         | Definición                                                                                  |
|---------------------------------|---------------------------------------------------------------------------------------------|
| DAD                             | Documento de Análisis y Diseño, describe cómo funciona un sistema por                       |
| RF                              | Requisito funcional, algo que el sistema debe hacer.                                        |
| RNF                             | Requisito no funcional, una condición de calidad que el sistema debe cumplir                |
| Bitácora                        | Registro histórico de eventos con fecha y hora, en este caso de cada alimento               |
| Etapas                          | Clasificación del ave según su edad: pollito o adulto.                                      |
| Tabla alimenticia parametrizada | Tabla con las cantidades de alimento recomendadas por tipo y etapa de alimentación          |
| Mobile-first                    | Enfoque de diseño donde la aplicación se construye pensando primero en dispositivos móviles |
| Inventario                      | Cantidad de alimento disponible en un momento dado, calculada a partir de la bitácora       |
| Proyección                      | Estimación de cuánto alimento se necesitará comprar en un periodo futuro                    |

## 8. Anexos

### 8.1. Mockups de pantallas

#### Login



#### Registro



#### Inicio

[Image Placeholder]

#### Mis aves

[Image Placeholder]

#### Alimentar

[Image Placeholder]

#### Inventario

[Image Placeholder]

#### Configuración

[Image Placeholder]